# AssetHub

VIZSGAREMEK

**Készítette: Princz Dániel Olivér, Baranyi-Bujdosó Gergő**

**2025**

# Tartalomjegyzék

**1. Bevezetés** **4**
1.1. Projekt célja 4
1.2. Téma választásának megindoklása 4
1.3. Célközönség 5
1.4. Program funkciók 5
1.5. Csoport tagjai és munkamegosztás 5
1.4.1 Munkamegosztás 5
**2. Fejlesztői dokumentáció** **6**
2.1. Fejlesztői környezet 6
2.1.1. API 6
2.2. Adatbázis 7
2.2.1. AuthLevel 8
2.2.2. ProductCategory 8
2.2.3. Store 9
2.2.4. Employee 9
2.2.5. StoreInventory 11
2.2.6. Product 12
2.2.7. Sales 12
2.3. Tesztelői fiók készítése 13
2.4. Fejlesztési lehetőségek 16
**3. Felhasználói dokumentáció** **17**
3.1. Mit lehet várni a programtól? 17
3.2. Hardver és Szoftver követelmények 17
3.2.1. Hardver követelmények 17
3.2.2. Szoftver követelmények 18
3.3. Program letöltése, telepítése és elindítása 18
3.4. Használati útmutató 22
3.4.1. Általános információ 22
3.4.2. Oldal specifikus információ 26
3.5. Hibajelzések 37
3.6. Mit lehet elérni Regisztráció nélkül? 38
3.7. Program jogkörök részletesen 38
3.8. Elérhetőségek információ kérés esetén 40

**4. Összefoglalás** **41**
4.1. Munkánk értékelése 41
4.2. Hogyan hasznosulhat a továbbiakban? 41
**5. Irodalmi Jegyzetek** **42**

# 1. Bevezetés

## 1.1. Projekt célja

A projekt elsődleges célja egy olyan virtuális rendszer megalkotása, amely lehetővé teszi, hogy a felhasználók egy könnyen kezelhető, interaktív, és több szakmához megfelelő applikációt kapjanak, ami összekötően tud mindenfajta bolti és áruházlánci feladatot megoldani, könnyebbíteni. A rendszer a világon minden cég-, áruház- és bolt tulajdonosnak megadja a lehetőséget, hogy egyetlen, közös program segítségével minden üzleti és adminisztrációs tevékenységet végezzenek, még az otthon kényelmében is.

A projekt két különlegességgel is szolgál: egy egyesített bolt kezelőfelület, ami rengeteg boltnak az adatát tudja egyszerre egy helyen tárolni egy cég vagy áruházlánc vezetőnek, és az áruházi készletről gombnyomásra készíthető excel fájl, amit később vissza lehet adni a programnak, hogy az információ segítségével töltse fel az árukészletet.

Emellett a letisztított kinézete és lényegre törő, egyszerűen kezelhető felhasználói felület miatt elkerülhető sok hiba, ami a bolt és adatbázis kárára menne, így segítve a fenntartásban.

## 1.2. Téma választásának megindoklása

Rengeteg bolt rengeteg különböző menedzseri felületet használ és lehetnek még olyan esetek is, hogy ugyanabban az áruházi láncban más felületet használnak, és pontosan ezt szeretnénk orvosolni egy minden befoglaló programba. Ráadásul problémát tudnak jelenteni az alkalmazottak számára, hogy ha valami változtatást igényel a rendszer, akkor mindenféleképpen át kell vinni az információt a főnökükhöz, mert csak ő tud hozzáférni. Ezért egy olyan központi alkalmazást hoztunk létre, ahol a már nem csak a főnök, hanem minden alkalmazott is tudja végezni a rászabott munkát még digitális formában is.

## 1.3. Célközönség

A fő célközönség nagy bolti vagy áruházilánci tulajdonosok, akiknek elég sok bolt után kell azonnal, több adatot elkérni, mint például az eladásokat, bevételt, kiadást és miegymást gyorsan és kényelmesen. Habár másodlagos célközönségként fel lehet sorolni szinte minden bolti vagy kis áruházlánci tulajdonost is, a program fő iránya sokkal jobban a nagy cégeknek fogja megteremteni a legtöbb hasznot, viszont nincs kizárva a későbbiekben a célközönséget kitágítani.

## 1.4. Program funkciók

A program funkció számban csekélyek, viszont annál fontosabbak a fő cél érdekében: áruházlánci menedzsment digitálisan a munkahely vagy otthon kényelméből. Minden oldal hasonló, de a témának és feladatnak megfelelő funkciókkal vannak ellátva, amiket az egyszerűség kedvéért kategóriákba lehet sorolni, programozási nyelv fordítással együtt: Felsorolás / GET lekérés, Hozzáadás / POST adat feltöltés, Szerkesztés / PUT, vagy PATCH adat változtatás, és Törlés / DELETE, avagy adat törlés. Az oldaloknak megfelelő és részletes leírást a 3. Felhasználói dokumentáció pont alatt lévő 3.4. Használati útmutatóban lehet megtalálni.

## 1.5. Csoport tagjai és munkamegosztás

A projektet Princz Dániel Olivér és Baranyi-Bujdosó Gergő készítette.

### 1.4.1 Munkamegosztás

**Baranyi-Bujdosó Gergő:** Frontend fő részei, Tesztelés és kód javítás nagyja

**Princz Dániel Olivér:** Backend fő részei, API készítése, Tervezői grafikonok

**Közös felelősség:** Backend és Frontend közös fejlesztései kisebb-nagyobb részekben, Dokumentáció elkészítése, Adatbázis felépítése, Tesztelés és Kód javítás kisebb-nagyobb részekben

# 2. Fejlesztői dokumentáció

## 2.1. Fejlesztői környezet

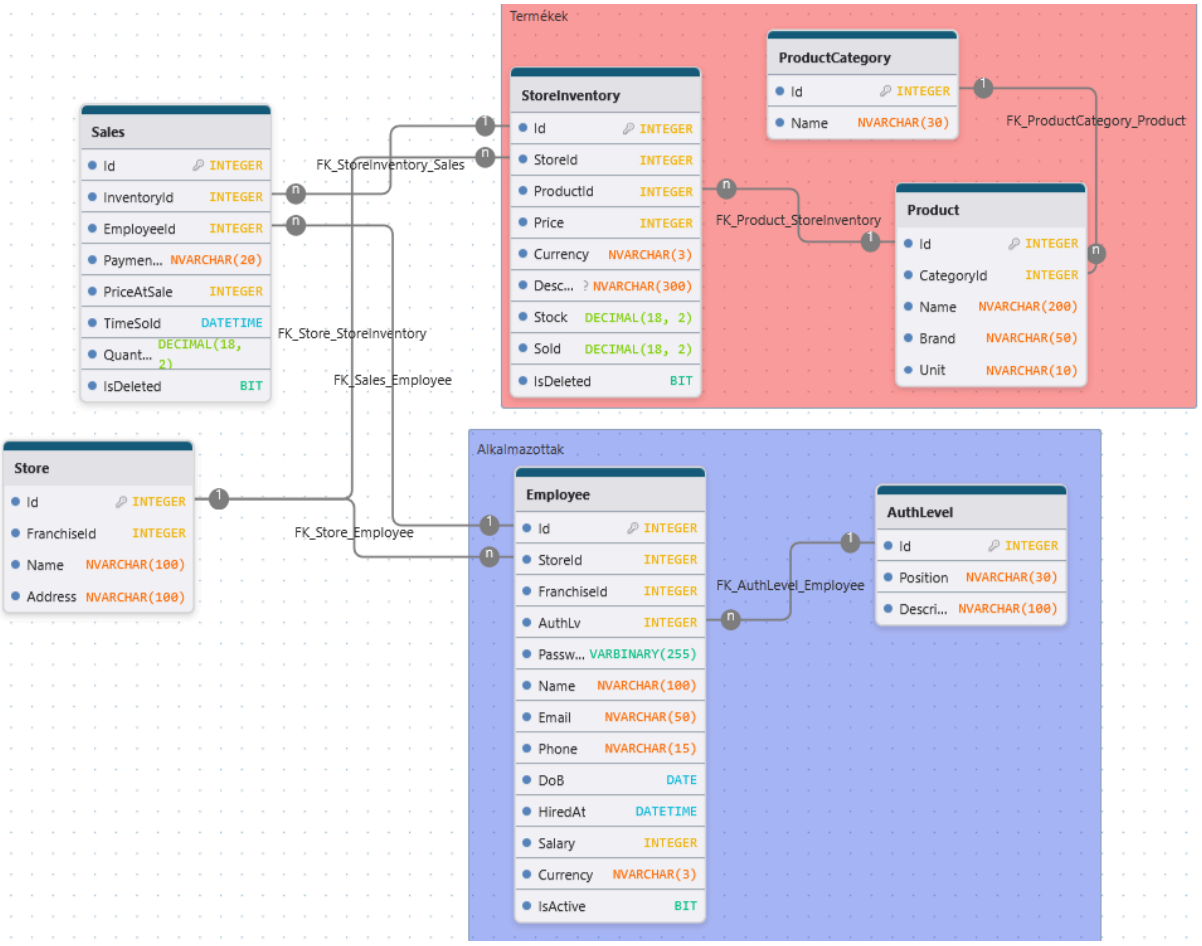
A projektet két különböző fejlesztői eszközzel és programnyelvben írtuk a egyszerűség és szintaktikai okok miatt. Továbbá három részre lehet osztani a program működését: **Frontend**, **Backend** és az **Adatbázis**.

A **Frontend**et, illetve a **Backend**et szintúgy a Microsoft által készített és karbantartott **Visual Studio Code** szoftverben készítettük, mert eddig ezzel volt a legtöbb tapasztalunk. A program **Frontend** részét a **TypeScript** nyelvű **Next.Js** keretrendszerben írtuk meg, mert számunkra a többi opción felül értelmezhetőbb az architektúrája és felépítése, illetve elég közel áll a HTML-hez, ami segítségével könnyebben át láthatjuk a két részt.

### 2.1.1. API

A **Backend** fő része a projektre szabott **JavaScript** nyelvű **Node.Js** keretrendszerű Application Programming Interface (API), ami lehetővé teszi az adatbázis szerver és a program közötti stabil kommunikációt. A látható SQL Query írás és könnyebben követhető kód szintaktika miatt választottuk a többi keretrendszer felett.

## 2.2. Adatbázis




(Készítettük a https://drawdb.vercel.app oldalon)

Az adatbázist a Microsoft által fejlesztett **MSSQL** programozási nyelvben készítettük el, amit a szintén Microsoft által karbantartott **SQL Server Management Studio 2021** segítségével tudtuk tesztelni és implementálni

**Mindegyik táblában van egy 32 bites egész számként tárolt 'Id' oszlop, ami az egyéni azonosítója minden sorra, és azonosításra és megkülönböztetésére szolgál. Ez az oszlop minden táblában (kivéve az AuthLevel táblában) 1-től kezdve automatikusan növekedni fog.**

### 2.2.1. AuthLevel

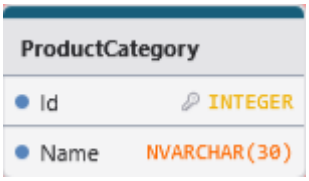
```
CREATE TABLE AuthLevel (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Position NVARCHAR(30) NOT NULL,
    Description NVARCHAR(100) NOT NULL
);
```

Egy statikus tábla a különböző autentikáció szintek tárolására. Az összes táblával ellentétben itt új adatot nem tud a felhasználó megadni, mert itt találhatók a különféle jogkörök a programon belül. Pl.: Admin, Áruházlánc tulajdonos, Menedzser, stb.

**Position**: A munkahelyi pozíció elnevezése. (Maximum 30 karakteres szöveg)

**Description**: A munkahelyi pozíció rövid leírása, ami továbbá a felhatalmazott jogosultságokat is tartalmazza az alkalmazott részére. (Maximum 100 karakteres szöveg)

### 2.2.2. ProductCategory




```
CREATE TABLE ProductCategory (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(30) NOT NULL
);
```

Egy tábla az eladásra kínált termékek kategóriákra, ami segít a termékek könnyebb szortírozásában. Az adatok hasonlóképpen működnek, mint a Product táblában található sablonokra.

**Name**: A kategória neve. (Maximum 30 karakteres szöveg)

### 2.2.3. Store

| Store | |
|---|---|
| Id | INTEGER |
| FranchiseId | INTEGER |
| Name | NVARCHAR(100) |
| Address | NVARCHAR(100) |

```sql
CREATE TABLE Store (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FranchiseId INT NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Address NVARCHAR(100) NOT NULL,
);
```

Az adatbázis fő táblája, ahol az áruházak és a bolti lánc fő adatait tároljuk.

**FranchiseId**: A Boltláncnak megfelelő azonosító. Új cég tulajdonos fiók készítésekor ez az azonosító felelős a általa készítette boltok összegzéséhez. (32 bites egész szám)

**Name**: Az áruház neve. (Maximum 100 karakteres szöveg)

**Address**: Az áruház címe. (Maximum 100 karakteres szöveg)

### 2.2.4. Employee

| Employee | |
|---|---|
| Id | INTEGER |
| StoreId | INTEGER |
| FranchiseId | INTEGER |
| AuthLv | INTEGER |
| Passw... | VARBINARY(255) |
| Name | NVARCHAR(100) |
| Email | NVARCHAR(50) |
| Phone | NVARCHAR(15) |
| DoB | DATE |
| HiredAt | DATETIME |
| Salary | INTEGER |
| Currency | NVARCHAR(3) |
| IsActive | BIT |

```sql
CREATE TABLE Employee (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    StoreId INT NOT NULL,
    FranchiseId INT NOT NULL,
    AuthLv INT NOT NULL,
    Password VARBINARY(255) NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(50) NOT NULL,
    Phone NVARCHAR(15) NOT NULL,
    DoB DATE NOT NULL,
    HiredAt DATETIME DEFAULT GETDATE(),
    Salary INT NULL,
    Currency NVARCHAR(3) DEFAULT 'HUF',
    IsActive BIT DEFAULT 1,
    CONSTRAINT FK_Store_Employee FOREIGN KEY (StoreId) REFERENCES Store(Id),
    CONSTRAINT FK_AuthLevel_Employee FOREIGN KEY (AuthLv) REFERENCES AuthLevel(Id)
);
```

Az alkalmazottak tábla, ami az alkalmazottak adatait tartalmazza.

**StoreId**: Összeköti az Employee és a Store táblát, megmondja hogy az alkalmazott melyik áruházban dolgozik. (32 bites egész szám)

**FranchiseId**: Ez az oszlop szolgál a különböző bolti láncok közötti megkülönböztetésre. Csak akkor kerül kitöltésre, ha cég tulajdonosként lép be valaki, különben ez a mező üres (NULL) lesz. (32 bites egész szám)

**AuthLv**: Az alkalmazott autentikáció szintje, amit a AuthLevel táblából adott opciók közül adjuk meg. (32 bites egész szám)

**Password**: A programba való bejelentkezéshez használt jelszava. Ez a mező nem egy konkrét szöveget, hanem egy kódolt sort tartalmaz nagyobb biztonság érdekében. (Bináris szám)

**Name**: Az alkalmazott neve. (Maximum 100 karakteres szöveg)

**Email**: Az alkalmazott email címe. (Maximum 50 karakteres szöveg)

**Phone**: Az alkalmazott telefonszáma. (Maximum 15 karakteres szöveg)

**DoB**: Az alkalmazott születési dátuma. (Egyszerű dátum formátum: yyyy-MM-dd)

**HiredAt**: Az alkalmazott felvételének dátuma. (Egyszerű dátum és idő formátum: yyyy-MM-dd hh-mm-ss-n*)

**Salary**: Az alkalmazott fizetése. (32 bites egész szám)

**IsActive**: Egy Igaz-Hamis oszlop, ami a fiók aktív státuszát mondja meg. Ha ennek az alkalmazott más táblában idegen kulcsként jelenik meg, ez az oszlop felel az adat megőrzéséért. (Bit/Boolean)

### 2.2.5. StoreInventory

| StoreInventory | |
|---|---|
| Id | INTEGER |
| StoreId | INTEGER |
| ProductId | INTEGER |
| Price | INTEGER |
| Currency | NVARCHAR(3) |
| Desc... | NVARCHAR(300) |
| Stock | DECIMAL(18, 2) |
| Sold | DECIMAL(18, 2) |
| IsDeleted | BIT |

```sql
CREATE TABLE StoreInventory (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    StoreId INT NOT NULL,
    ProductId INT NOT NULL,
    Price INT NOT NULL,
    Currency NVARCHAR(3) NOT NULL,
    Description NVARCHAR(300) NULL,
    Stock DECIMAL(18, 2) NOT NULL,
    Sold DECIMAL(18, 2) DEFAULT 0.00,
    IsDeleted BIT DEFAULT 0,
    CONSTRAINT FK_Store_StoreInventory FOREIGN KEY (StoreId) REFERENCES Store(Id),
    CONSTRAINT FK_Product_StoreInventory FOREIGN KEY (ProductId) REFERENCES Product(Id)
);
```

Az áruház árukészlet adatainak gyűjteménye.

**StoreId**: Kapcsolatot teremt a Store táblával, meghatározva, hogy melyik áruház készletét kell betölteni. (32 bites egész szám)

**ProductId**: Kapcsolatot teremt a Product táblával, így megtudva, hogy melyik terméknek az adatait kell betölteni. (32 bites egész szám)

**Price**: A termékek ára. (32 bites egész szám)

**Description**: A termék leírása rövid leírása. (Maximum 300 karakteres szöveg)

**Stock**: Árukészleten lévő termék mennyisége. (18 számból álló, századra pontos decimális szám)

**Sold**: Egy adott termékre vonatkozó egységből eladott mennyiség. (18 számból álló, századra pontos decimális szám)

**IsDeleted**: Egy Igaz-Hamis oszlop, ami a készleten lévő termék aktív státuszát mondja meg. Ha ennek ez az adat más táblában idegen kulcsként jelenik meg, ez az oszlop felel az adat megőrzéséért. (Bit/Boolean)

### 2.2.6. Product

| Product | |
|---|---|
| Id | INTEGER |
| CategoryId | INTEGER |
| Name | NVARCHAR(200) |
| Brand | NVARCHAR(50) |
| Unit | NVARCHAR(10) |

```sql
CREATE TABLE Product (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    CategoryId INT NOT NULL,
    Name NVARCHAR(200) NOT NULL,
    Brand NVARCHAR(50) NOT NULL,
    Unit NVARCHAR(10) NOT NULL,
    CONSTRAINT FK_ProductCategory_Product FOREIGN KEY (CategoryId) REFERENCES ProductCategory(Id)
);
```

Itt tároljuk az úgynevezett ‘termék sablonokat’, amit később a StoreInventory tábla segítségével többször is fel tudunk használni adat ismétlés nélkül.

**CategoryId**: Összeköti a Poduct és a ProductCategory táblákat, így kategorizálni lehet a hasonló termékeket. (32 bites egész szám)

**Name**: A termék neve. (Maximum 200 karakteres szöveg)

**Brand**: A termék márkája. (Maximum 50 karakteres szöveg)

**Unit**: A termék mértékegysége és/vagy mennyiség. (Maximum 10 karakteres szöveg)

### 2.2.7. Sales

| Sales | |
|---|---|
| Id | INTEGER |
| InventoryId | INTEGER |
| EmployeeId | INTEGER |
| Paymen... | NVARCHAR(20) |
| PriceAtSale | INTEGER |
| TimeSold | DATETIME |
| Quant... | DECIMAL(18, 2) |
| IsDeleted | BIT |

```sql
CREATE TABLE Sales (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    InventoryId INT NOT NULL,
    EmployeeId INT NOT NULL,
    PaymentMethod NVARCHAR(20) NOT NULL,
    PriceAtSale INT NOT NULL,
    TimeSold DATETIME DEFAULT GETDATE(),
    Quantity DECIMAL(18, 2) NOT NULL,
    IsDeleted BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_StoreInventory_Sales FOREIGN KEY (InventoryId) REFERENCES StoreInventory(Id),
    CONSTRAINT FK_Employee_Sales FOREIGN KEY (EmployeeId) REFERENCES Employee(Id)
);
```

A Sales tábla az eladásokat tárolja és a hozzátartozó fontos adatokat.

**InventoryId**: Összeköti a Sales táblát az StoreInventory táblával, amivel meg tudjuk határozni a termék minden adatát, beleértve, hogy el lehet-e adni belőle a megadott mennyiséget. (32 bites egész szám)

**EmployeeId**: Az eladást végző alkalmazott Id-je. (32 bites egész szám)

**PaymentMethod**: Az eladásnál használt fizetési eszköz. (Maximum 20 karakteres szöveg)

**PriceAtSale**: A termék ára az eladáskor. (32 bites egész szám)

**TimeSold**: Az eladás dátuma. (Egyszerű dátum és idő formátum: yyyy-MM-dd hh-mm-ss-n*)

**Quantity**: Termékből eladott mennyiség. (18 számból álló, századra pontos decimális szám)

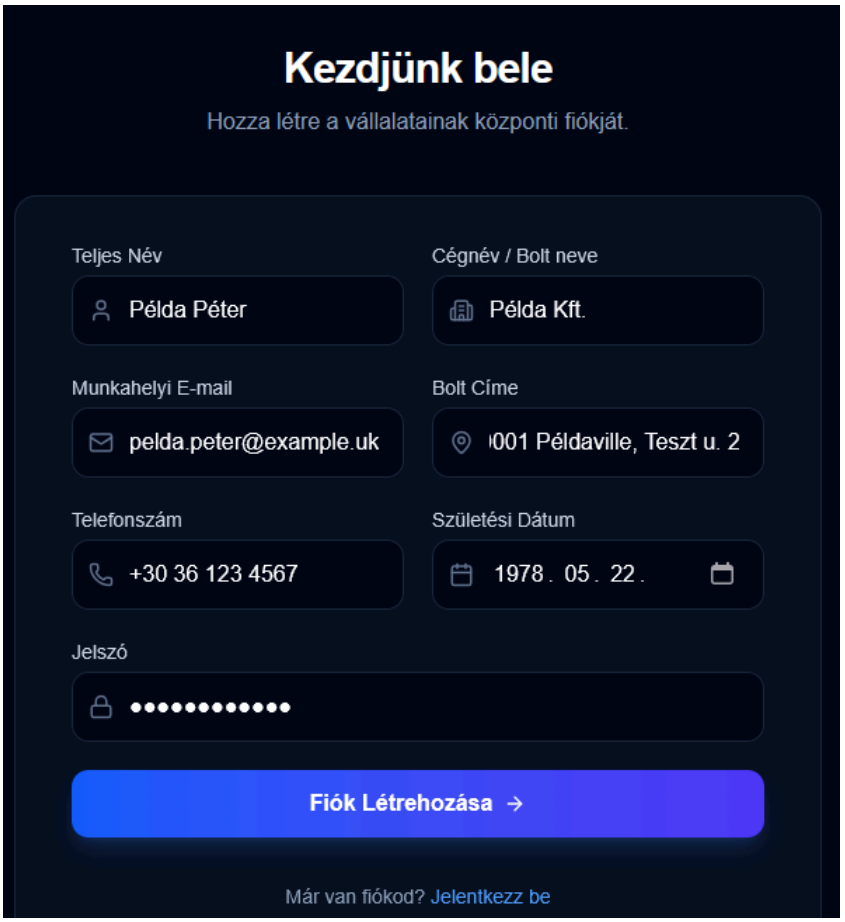
**IsDeleted**: Egy Igaz-Hamis oszlop, ami az eladási adat aktív státuszát mondja meg. Ha ennek az eladásnak más táblában idegen kulcsként jelenik meg, ez az oszlop felel az adat megőrzéséért. (Bit/Boolean)

## 2.3. Tesztelői fiók készítése

Külön tesztelői fiókot habár nem tudunk biztosítani a jelszó típus váltás miatt, egy egyszerű bemutatót a fiók készítéshez tudunk szolgálni:

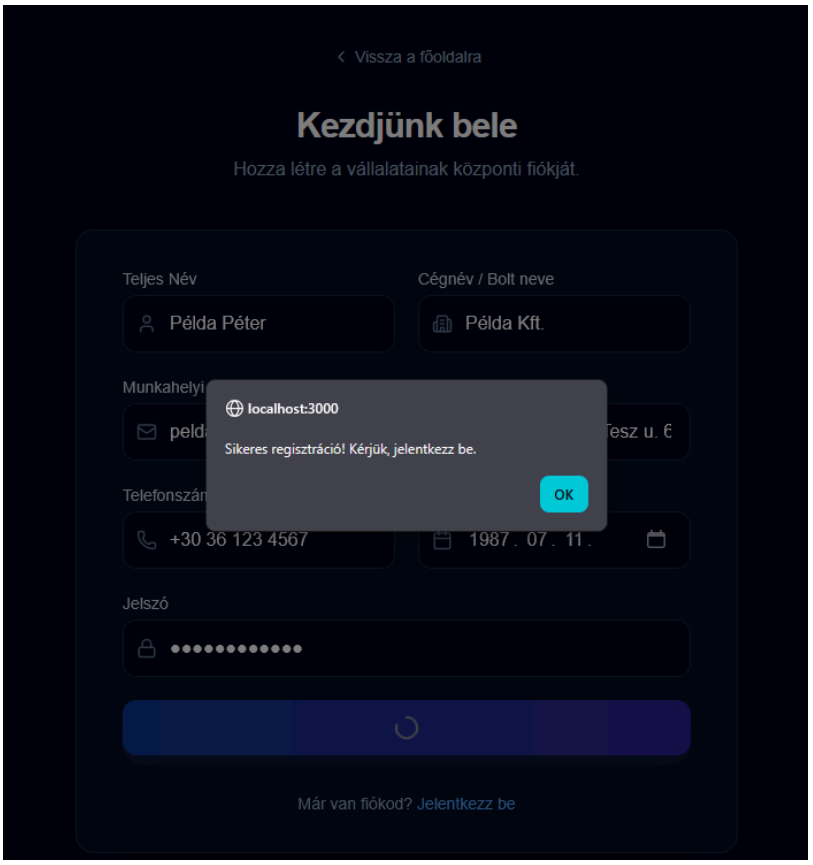
1., Navigáljunk a ‘Regisztráció’ oldalra:

2., Írjuk be minden adat mezőhöz a releváns információkat.

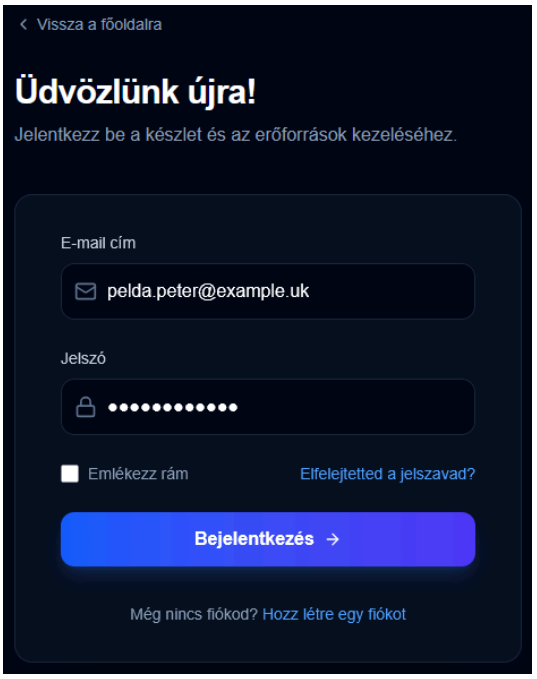



3., Miután ezt megtettük, nyomjunk rá a ‘Fiók Létrehozása’ gombra a fiók létrehozásához.

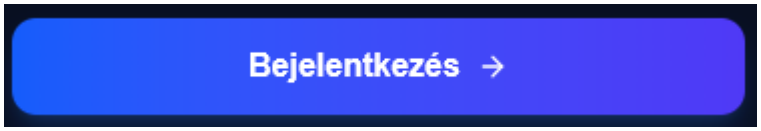
4., Ha mindent megfelelően adtunk meg, akkor ezt az üzenet kell hogy kapjuk és átleszünk irányítva a ‘Bejelentkezés’ oldalra.




5., Adjuk meg az előbb beírt releváns adatok (Email cím és jelszó) a bejelentkezéshez.



6., Ezután nyomjunk rá a ‘Bejelentkezés’ gombra és újonnan át lesznek irányítva a program Vezérlőpultjához! Gratulálunk, ezzel elkészítette saját (vagy tesztelői) fiókot!




**Fontos tudnivaló**: A regisztráció mindig úgy fogja felvenni a fiókot, mintha ön lenne egy áruházláncnak a tulajdonosa. Soha nem fog más jogkört megadni a fióknak, ha mégis, kérjük keressenek az elérhetőségek alatt.

## 2.4. Fejlesztési lehetőségek

A programunkban az egyik legnagyobb siker a tömörtelen mennyiségű fejlesztési lehetőség, amit enyhe túlzással végtelenül tud fejlődni a megfelelő kezekben. Kilistázni az összes lehetőséget nagyon hosszú és időigényes feladat lenne, szóval áttérnék inkább a legjobb módokat szeretném felsorolni, hogy mit lehetne még tenni a program fejlesztése érdekében:

- Integráció beszállító cégek oldalaival, program felületeivel az áru megrendelések érdekében anélkül hogy elhagynánk a programot.
- Interaktív 2D Drag-and-Drop térkép a boltról a rakodók és polc feltöltőknek, amit egyszerűen és gyorsan lehet módosítani és kinyomtatni.
- Integráció más boltban használható hardverrel és szoftverrel, például kasszákkal, hogy automatikusan frissüljön az adatbázis és könnyebben tudjuk nyilvántartani a bevételt.
- Statisztika az bolti áru fogyásáról, válaszolva ezekre a kérdésekre: Mi fogy a legjobban? Milyen márkát szeretnek az vásárlók? Mennyire volt a magasabb ár hatással a vásárlókra?

# 3. Felhasználói dokumentáció

## 3.1. Mit lehet várni a programtól?

Definiálva, a program egy komplex áruházlánc és bolt kezelő és adminisztrációs rendszer, különböző jogkörökkel és egyszerűen, gombnyomásokkal kezelhető felhasználói felülettel. Az előbb említett jogkörök hierarchikusan van felépítve (avagy Piramis szerűn), így egyszerűen a jogkörök számát a megadott lehetőségekkel - fordítottan - össze lehet hasonlítani. Továbbá, minden egyes oldalt úgy terveztünk, hogy szinte sehol se kelljen 2-nél több kattintás a kezeléséhez, így gyorsan, könnyen és áramvonalasan lehet használni mint tulajdonosként, üzletvezetőként vagy esetleg még alkalmazottként is.

Tömören összefoglalva egy olyan program van a felhasználó kezében, ami képes üzletlánci és bolti nyilvántartási feladatok megoldani a munkahelyen vagy az otthon kényelméből.

## 3.2. Hardver és Szoftver követelmények

**Fontos tudnivaló**: Ezt a programot 2026.03.31., ezt a dokumentációt 2026.03.29.-n lett írva. Az itt leírt követelmények ezt fogják bemutatni. A jövőben ezek a követelmények **változni fognak**!

### 3.2.1. Hardver követelmények

(Ezek a követelmények főleg a programot futtató rendszer / gép / szervernek szól. Maga a weboldal amire csatlakozik nagyon kevés erőforrást igényel.)

A program futtatásához elsősorban egy modern 64 bites processzor szükséges (x64 vagy ARM64, pl.: Apple M1/M2/M3), minél több maggal a gyorsabb futtatás érdekében, viszont a magok száma másképp nem befolyásolják a működést.

Ezt követően legalább 4Gb, de jobban ajánlott 8+ Gb Memóriát adni a program futtatásához és legalább 500Mb - 1Gb tárhelyet lefoglalni a szoftver csomagok miatt.

### 3.2.2. Szoftver követelmények

Az program a legújabb **Node.Js** keretrendszer verziót használja mint a program “motorja”, ezért feltétlenül le kell tölteni a **Node.Js** fejlesztői által készített csomagokat és magát a keretrendszer programját. Felhasználóként elég a 20.9 verziót letölteni, viszont mivel hamarosan a 20.x verzió a életciklus végét járja 2026 Áprilisán, ezért érdemesebb minimum a **22.x** verziót vagy **24.x** verziót letölteni a biztonság kedvéért.

Ahhoz hogy fusson maga a **Node.Js** szoftvere, az operációs rendszernek meg kell felelnie a következő specifikációknak:

- Microsoft Windows 10 vagy 11+
- Apple MacOS 13.5+
- Linuxnak bármely modern disztribúciója (pl.: Ubuntu 22.04+, Debian 11+, stb.)

Külön figyelmet kíván hogy a program csak bizonyos böngésző verziókon túl tud csak működni. A böngészők és minimum verzió számuk a következő:

- Google Chrome 111+
- Microsoft Edge 111+
- Mozilla Firefox 111+
- Apple Safari 16.4+

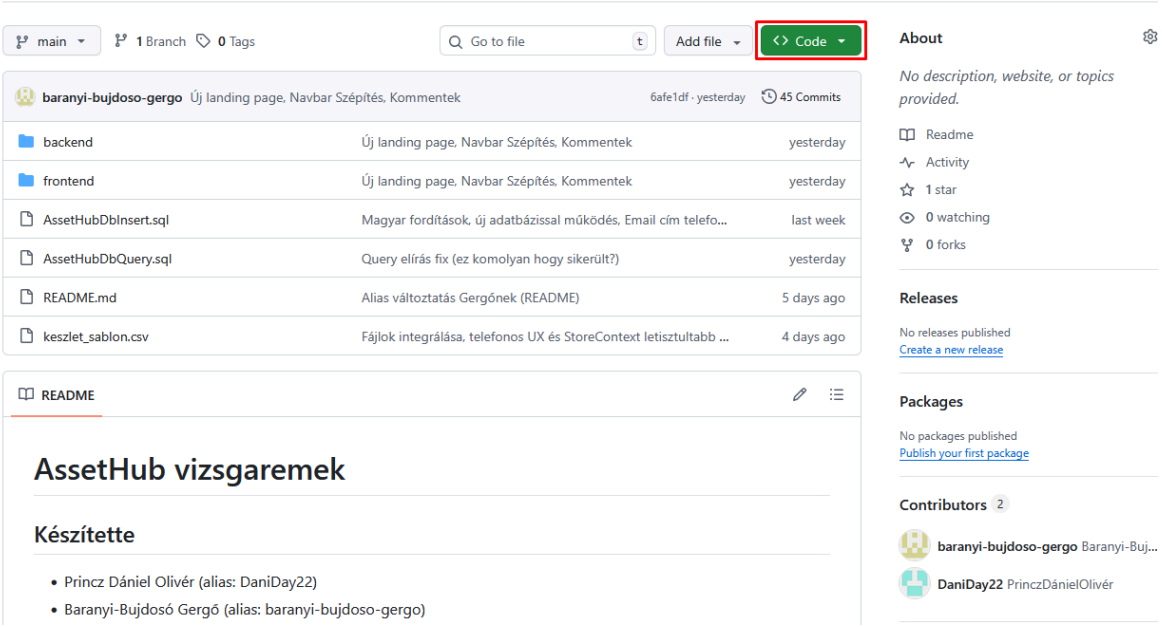
Utoljára, szükség lesz a **Node.Js** által fejlesztett *Node Package Manager*re, amit a 3.3.-s pontban lesz részletezve.

## 3.3. Program letöltése, telepítése és elindítása

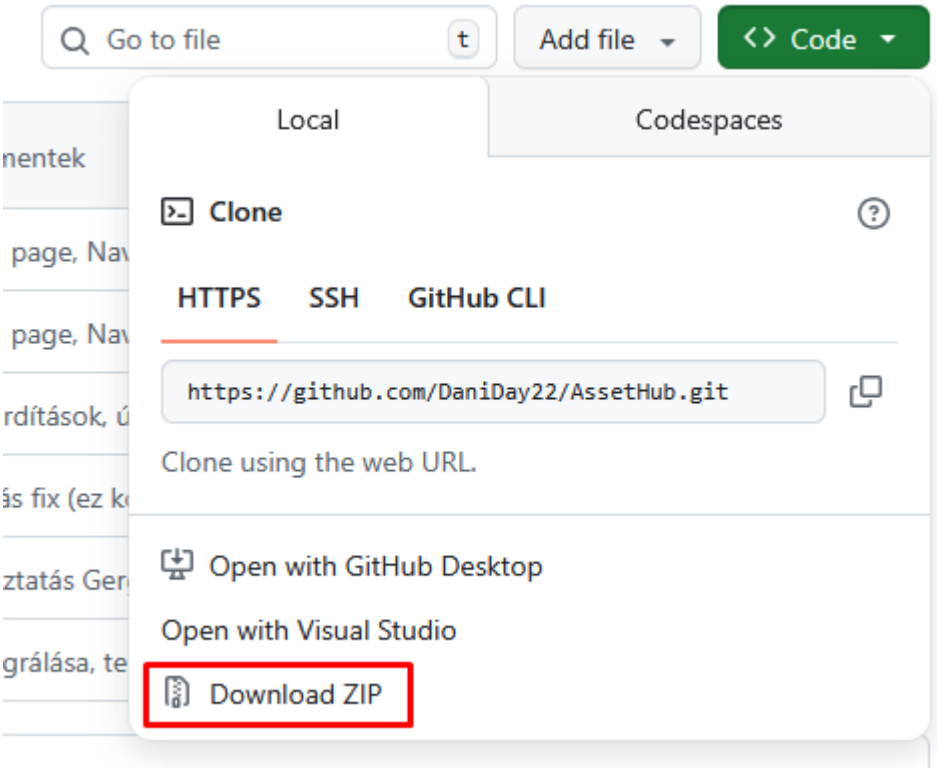
A programot a következő GitHub linkről tudjuk letölteni teljesen ingyen .zip fájlként vagy külön a parancssor / Windows Powershell segítségével (Regisztráció lehet hogy szükséges lesz): https://github.com/DaniDay22/AssetHub.git

.Zip fájl letöltése:

1., Navigáljunk a projekt GitHub oldalára és kattintsunk rá a ‘Code’ gombra.

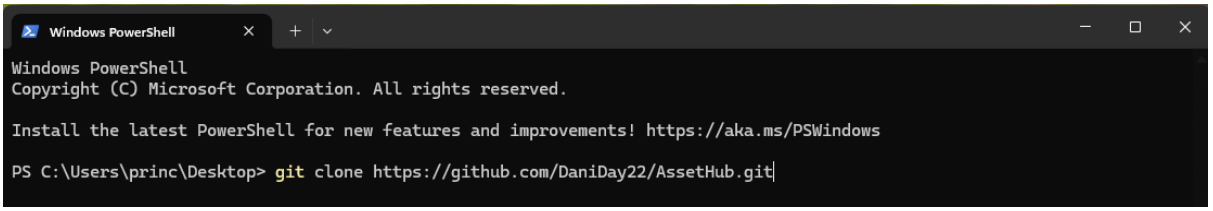



2., Kattintsunk rá a ‘Download ZIP’ gombra és az oldal letölti a programot számodra.



Konzolos letöltés:

1., Nyissuk meg a Parancssort vagy Windows Powershell-t és írjuk be a következő sort, majd egyszerűen nyomjuk meg az Enter-t, és letölti a projekt mappát.



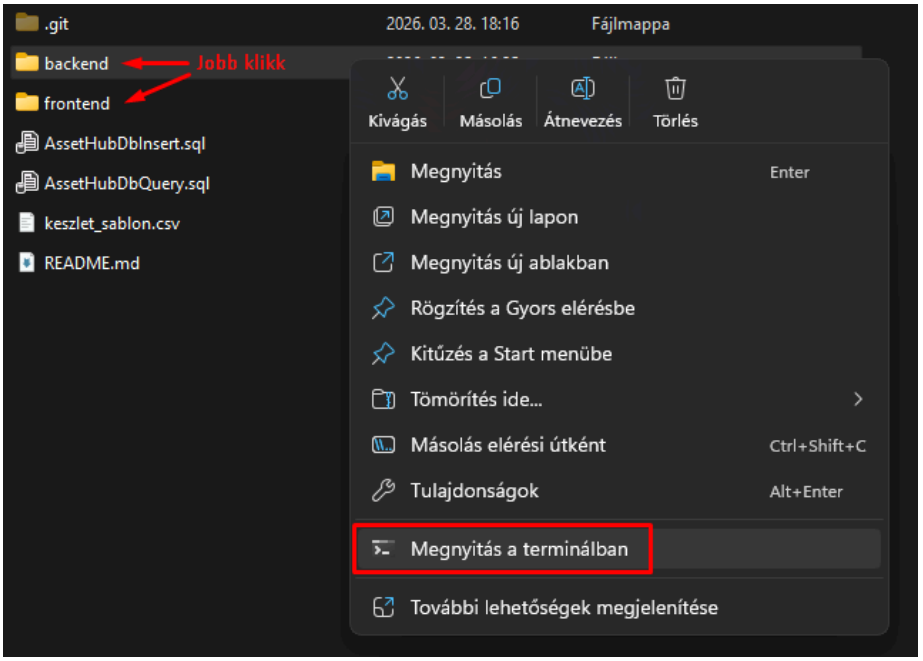

Miután ez meg van, térjünk át a **Node.Js** letöltésére, ami a program keretrendszerének kell. A folytatáshoz látogasson el a következő linkre: https://nodejs.org/en, és a vörös téglalappal kijelölt gombra kattintva töltsük le a megfelelő letöltőt.

Mielőtt belekezdenénk a **Node.Js** általi parancsok kiadásához, találni fogsz 2 .sql kiterjesztésű fájlt, amit a megfelelő SQL szerver kezelő programba kell lefuttatni. A sorrend a két fájl között számít!

Először töltsük fel és futtassuk le az ‘AssetHubDbQuery’ nevű fájlt, ami tartalmazza az adatbázist és a táblákat elkészítő parancsokat. Ha ez már megtörtént, akkor futtassuk le az ‘AssetHubDbInsert’ nevű fájlt, mert ez tartalmazza a különböző szerepköröket.

Ha le lett töltve a **Node.Js** és készen áll az SQL adatbázis, akkor navigáljunk el a projekt mappájába és kövesd a következő lépéseket:

1., Jobb kattintással nyissuk meg a ‘frontend’ és ‘backend’ mappákat a Parancssorban




2., A Parancssorba adjuk ki a következő parancsokat:

- **Backend**nél: **npm start**, vagy **node server.js**
- **Frontend**nél: **npm run dev**

Ha minden jól sikeredett, akkor a következő üzenetek kell kapni (**Fontos tudnivaló**: A program használata közben a két konzolt **ne zárja be**, mert különben a program az a része is bezárul):

**Backend**

```
PS D:\Programozás\Vizsga\AssetHub\backend> npm start

> backend@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (7) from .env -- tip: ⚙ write to custom object with { processEnv: myObject }
[dotenv@17.3.1] injecting env (0) from .env -- tip: 🤖 agentic secret storage: https://dotenvx.com/as2
Backend fut: http://localhost:5000
API: http://localhost:5000/api
```

**Frontend**

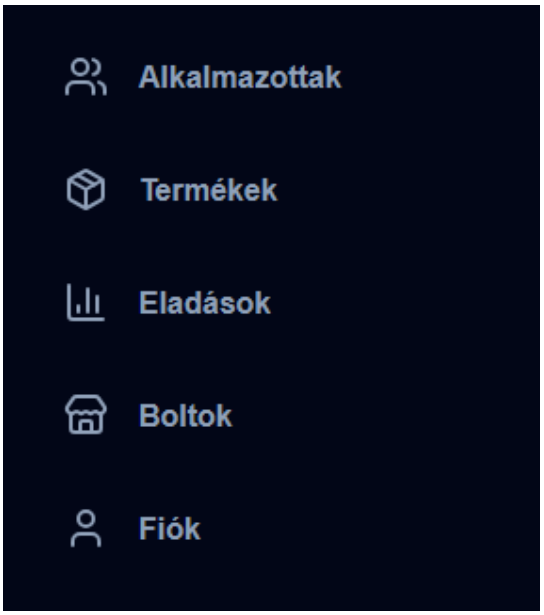
```
PS D:\Programozás\Vizsga\AssetHub\frontend> npm run dev

> frontend@0.1.0 dev
> next dev

▲ Next.js 16.1.6 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://192.168.1.67:3000

✓ Starting...
⚠The "middleware" file convention is deprecated. Please use "proxy" instead. Learn more: https://nextjs.org/docs/messag
es/middleware-to-proxy
✓ Ready in 699ms
```

A program eléréséhez / megnyitásához nyomjunk egy CTRL + D a Local linkre, vagy a Network linkre a Frontend résznél. Ha mindent jól csináltunk, akkor a kezdőlapra fog irányítani a böngésződben.

## 3.4. Használati útmutató

### 3.4.1. Általános információ

A program (a dokumentáció írásakor) 4, üzletlánc tulajdonosként 5 különböző menüponttal rendelkezik a következőképpen elrendezve, figyelembe véve, hogy jogkörök alapján ezekben a menüpontokban az adatok változhatnak. Üzletlánc tulajdonosként boltonként is megtekinthető a következő menüpontok adatai:




- **Alkalmazottak**: Egy bizonyos bolt alkalmazotti listája.
- **Termékek**: Egy bizonyos bolt terméklistája
- **Eladások**: Egy bizonyos bolt eladásainak gyűjteménye. Itt található a bevétel és a kiadások száma is.
- **Boltok** (üzletlánc tulajdonosnak): Az üzletláncban található boltok gyűjteménye
- **Fiók**: A fiók adatai és ahol különböző adatok lehet megváltoztatni, pl.: Telefonszám, Email, Jelszó.

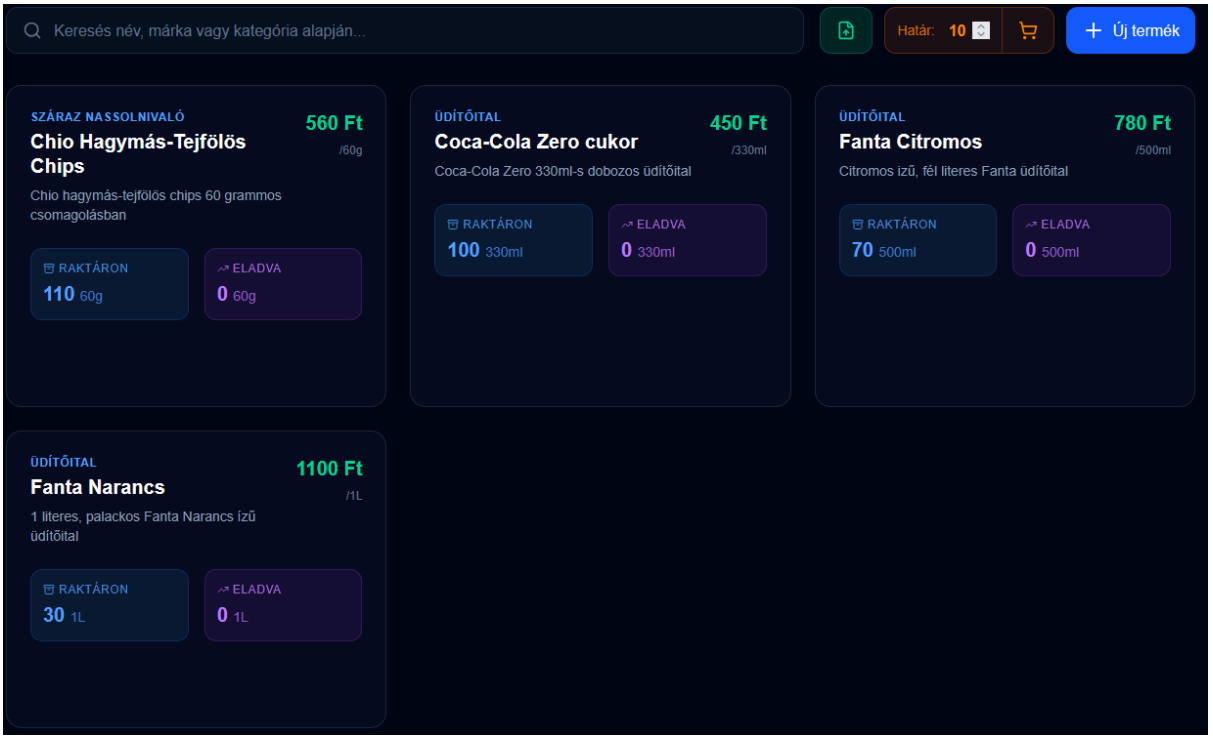
Mielőtt rátérnénk az összes menüpont részletes bemutatásához, előbb bemutatnám az összes menüpont (kivéve Fiók menüpont) közötti két hasonlóságot. Kezdve, hogy mindegyik tartalmaz egy keresősáv, amibe ha beleírunk valamit, akkor a menüpontnak megfelelően kiszűri a beírt szöveget tartalmazó adatokat.

Példa a Termékek menüpontból:

Egy keresősáv hasonlóan fog kinézni ehhez a sávhoz, keresse a nagyító ikont és megtalálja. A keresősávban található halvány jelölt példa mutatja, hogy mi alapján tudunk szűrni.

Ebben az esetben név, márka vagy kategória alapján fogunk tudni a termékek közül válogatni. Így fog kinézni egy szűrés:

- A teljes lista, nincs beírt tartalom a keresősávba:

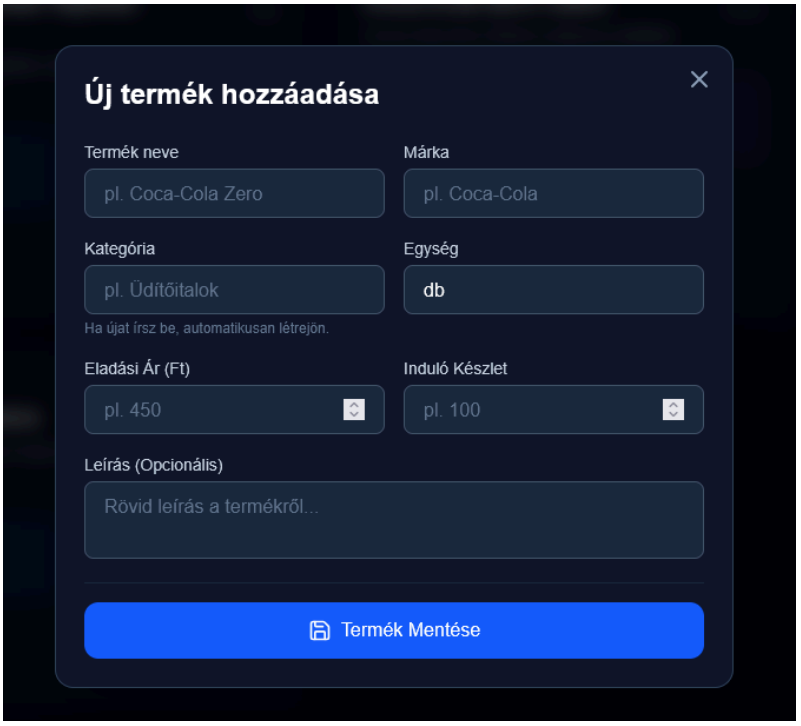


- Szűrés márka alapján:

- Szűrés név alapján; Szűrés kategória alapján:

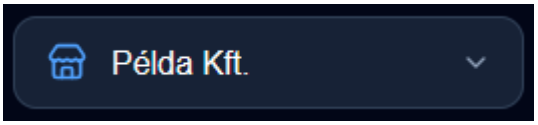
A másik funkció, ami mind a 4 oldalon megtalálható egy kék '+' szimbólum gomb, ami az oldalakkal eltérően változhat szöveg kiegészítéssel, viszont az mindegyik ugyanazt a célt szolgálja; Valami új dolgot adunk hozzá egy gyűjteményhez. Vegyük elő újra a Termékek oldalt példának:

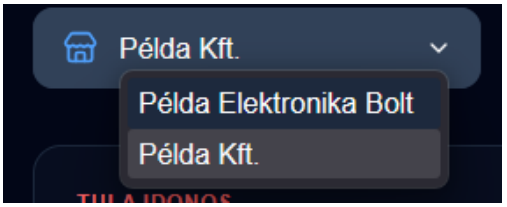
Erre a gombra rányomva megjeleníti a oldalon lévő specifikus felugró ablakot, ami külön-külön adatok fog kérni a felhasználótól. A Termékek oldal felugró ablaka például így néz ki:




A felugró ablakon lévő összes mezőt - kivéve ahol zárójelek közt ki van írva, hogy 'Opcionális' - ki kell tölteni az megfelelő adatokkal. Fontos tudni, hogy az adatot csak akkor menti el az oldal az adatbázisba, ha megnyomva a kék 'Termék Mentése' gombot.

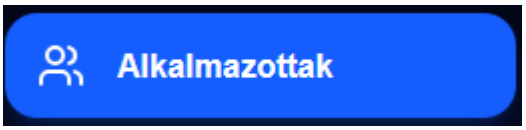
Továbbá, üzletlánc tulajdonosi fiókkal, és olyan üzletlánccal, ami 2 vagy több boltot von magába ezzel a lenyitható gombbal lehet váltogatni a boltok között:

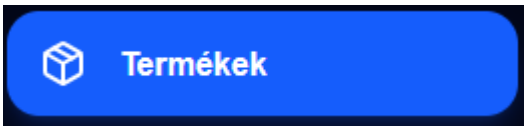


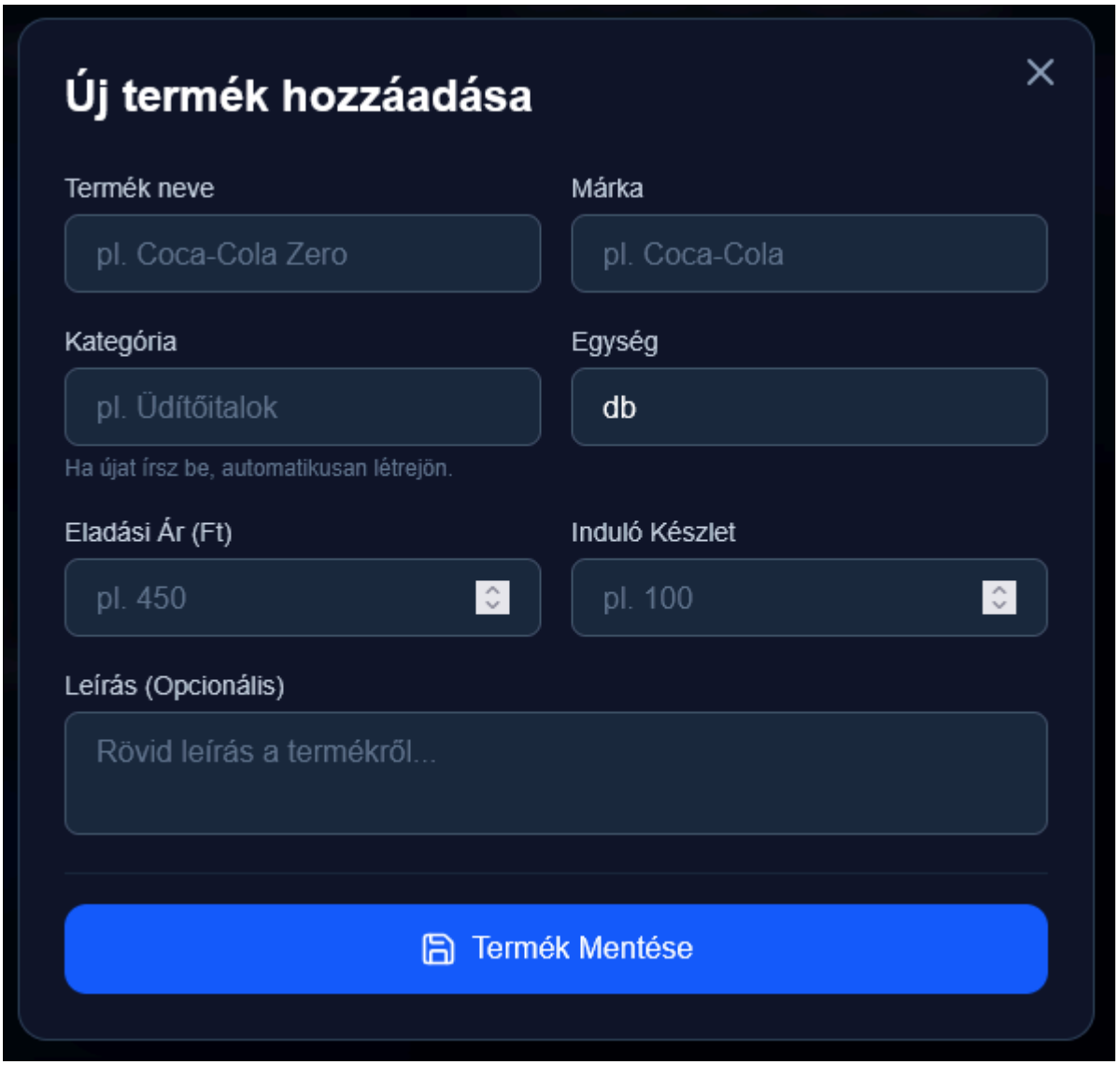



### 3.4.2. Oldal specifikus információ

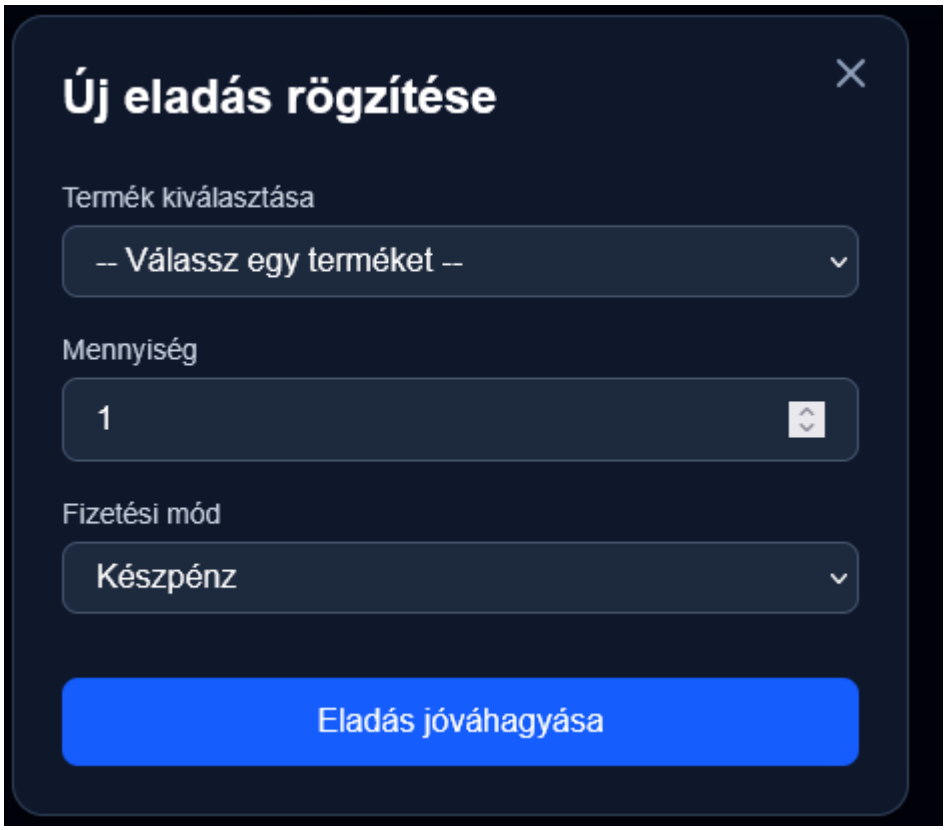
Habár a keresősáv és a '+' szimbólumos hozzáad gomb az mind a 4 kezelő oldalon megtalálható, vannak alapvető különbségek és eltérések köztük. Mivel ezt az általános információ (3.4.1.) résznél már elmagyaráztam, bemutattam, ezért helyette csak képekkel fogok szolgálni és jelölni fogom, melyik oldalról van szó (további információt adni fogok, ahol szükséges):



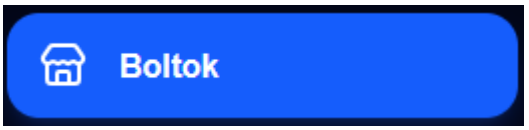

Termékek

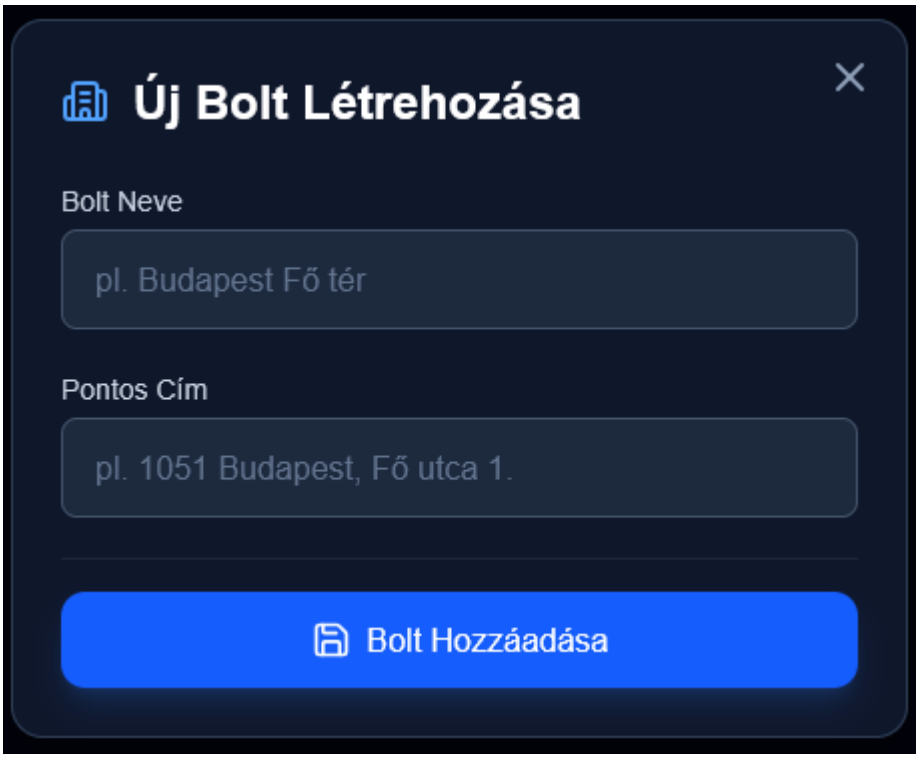

Új termék hozzáadása
Termék neve
pl. Coca-Cola Zero
Márka
pl. Coca-Cola
Kategória
pl. Üdítőitalok
Egység
db
Ha újat írsz be, automatikusan létrejön.
Eladási Ár (Ft)
pl. 450
Induló Készlet
pl. 100
Leírás (Opcionális)
Rövid leírás a termékről...
Termék Mentése

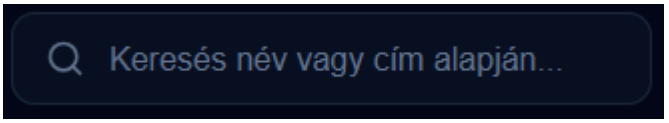
Keresés név, márka vagy kategória alapján...




(A keresősáv teljes szövege: “Keresés termék, márka vagy eladó **(neve) alapján**”)



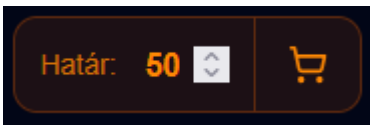

Folytatva, az **'Alkalmazottak'** oldalon az Tulajdonos és Üzletvezető jogkörrel minden olyan alkalmazottnak az adatát lehet megváltoztatni (jelszó kivételével), vagy törölni, akik a program jogkörök részletesen (3.6.) pont alatt található **ID-nek a száma nagyobb, mint a tiéd**. Saját adataidat nem lehet itt megváltoztatni, ahhoz kérjük forduljanak a 'Fiók' oldalhoz. Példa:

Bejelentkezett fiók: Példa **Péter** (Tulajdonos)

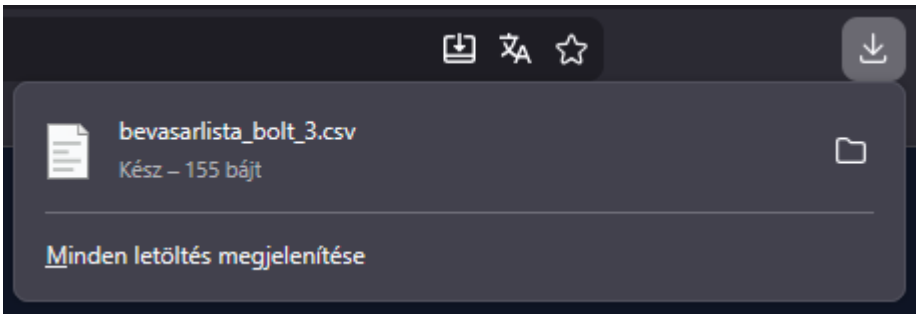
Bejelentkezett fiók: Példa **Parker** (Üzletvezető)

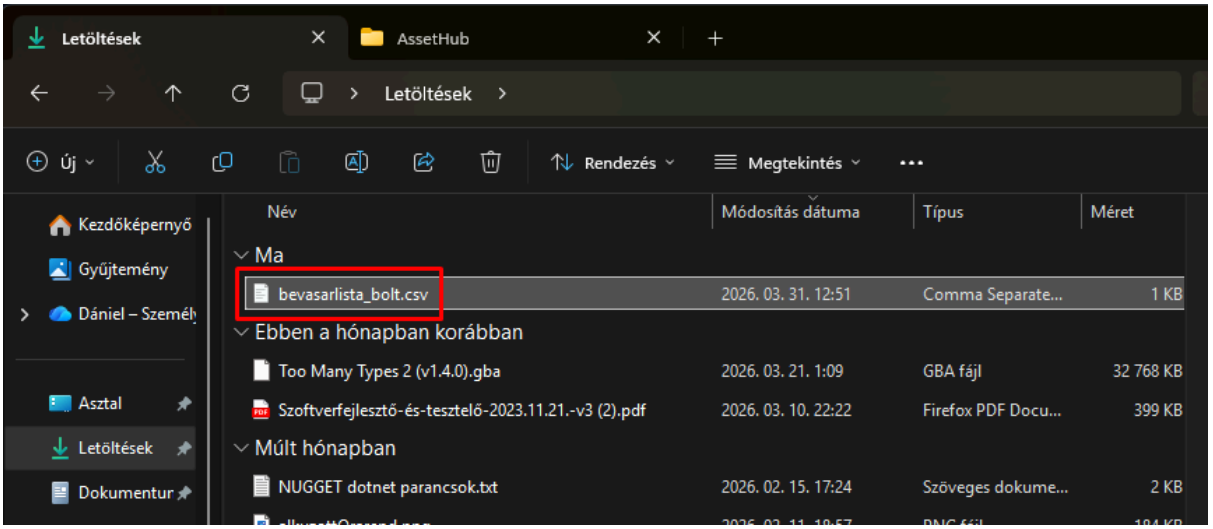
A **'Termékek'** oldalon lehet talán a legtöbb sajátosságot megtalálni a programban. Kezdve egy apróséggal, hogy a 'Szerkesztés' gomb és törlés (kuka ikon) gomb csak akkor jelenik meg, ha ráhúzzuk az egeret.

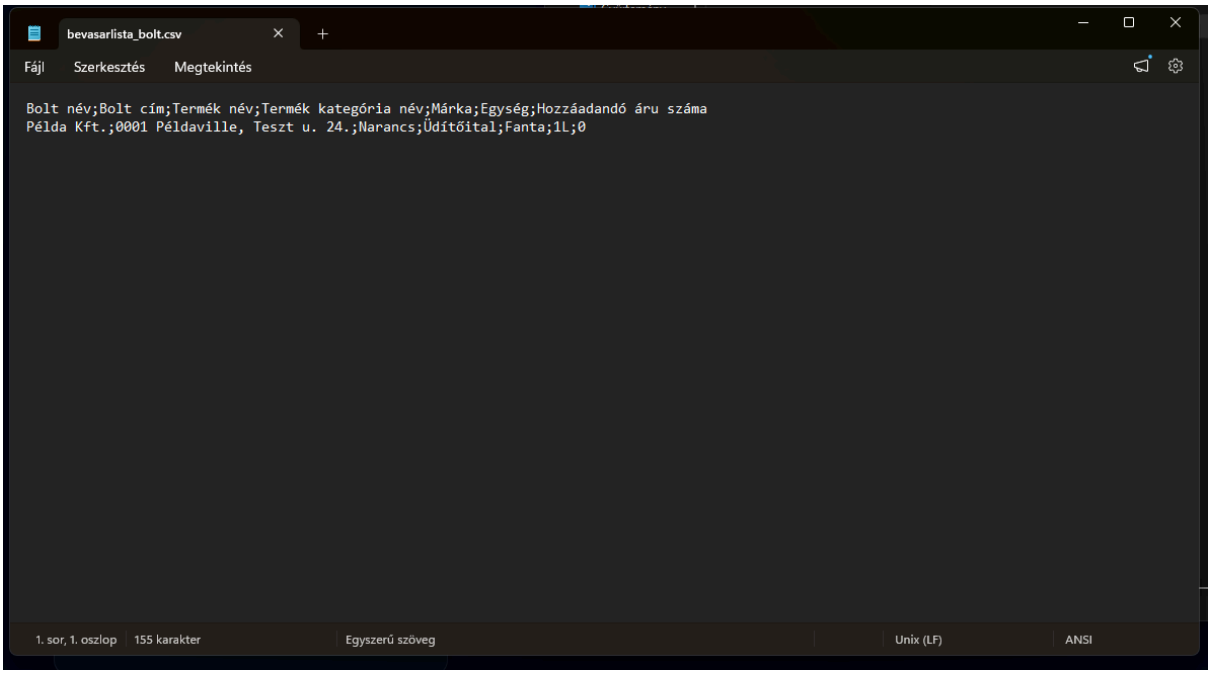
A legfontosabb innováció itt található meg, egy **CSV készítő és betöltő**, ami a meghatározott küszöb alatt kigyűjti minden olyan terméknek az adatai és boltnak az adatait, ahol éppen valamilyen szinten készlet hiány van. A készítő gomb így néz ki:



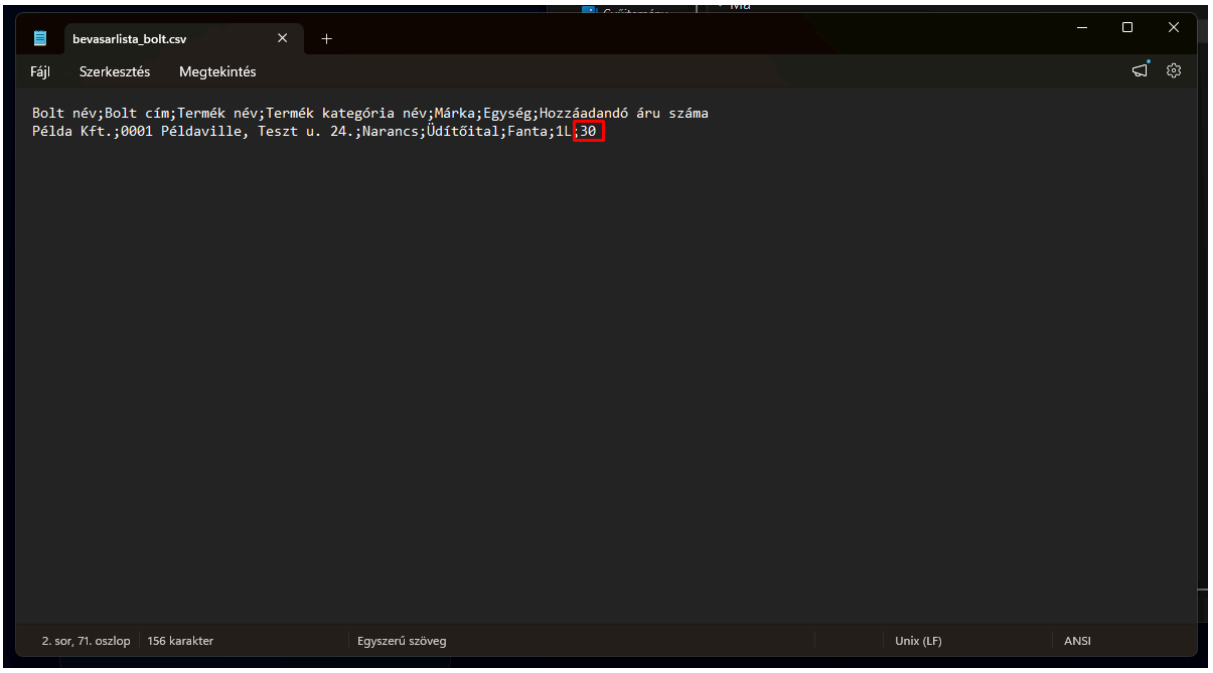
A bevásárlókocsi ikonra kattintva letölti a személynek a megfelelő sablonú .csv kiterjesztésű fájlt a határ alatti termékekről:



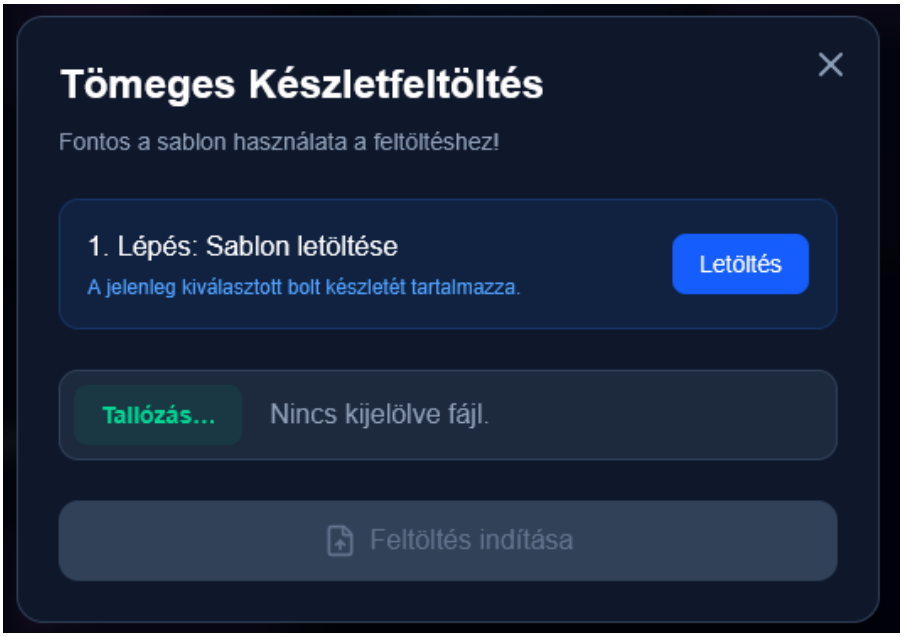



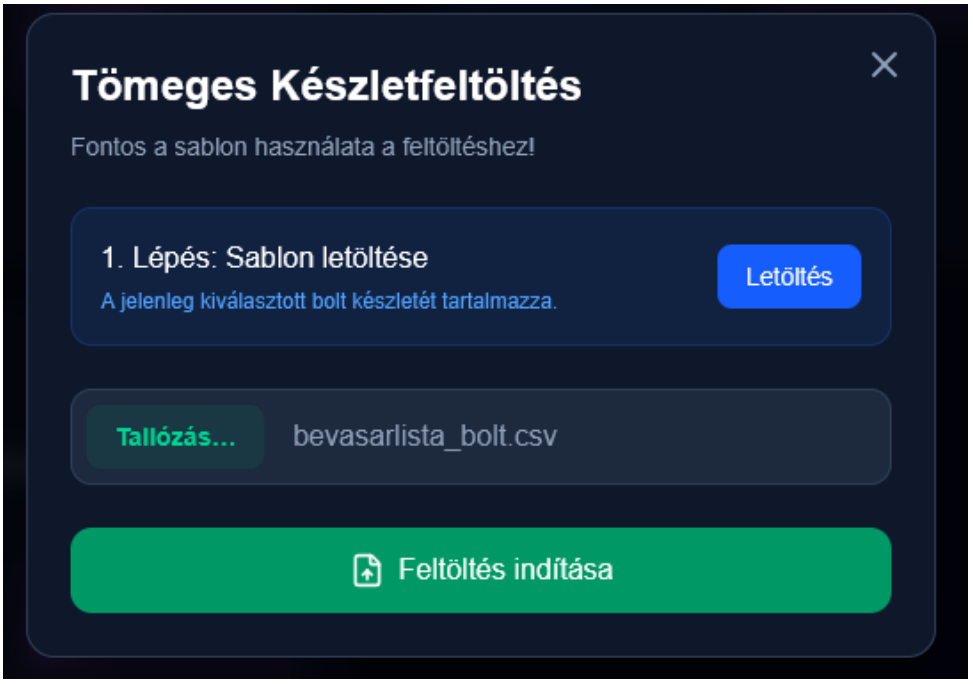
A legjobboldalibb oszlopot megváltoztatva lehet megmondani majd a CSV feltöltőnek, hogy mennyit terméket akar feltölteni a készletébe:



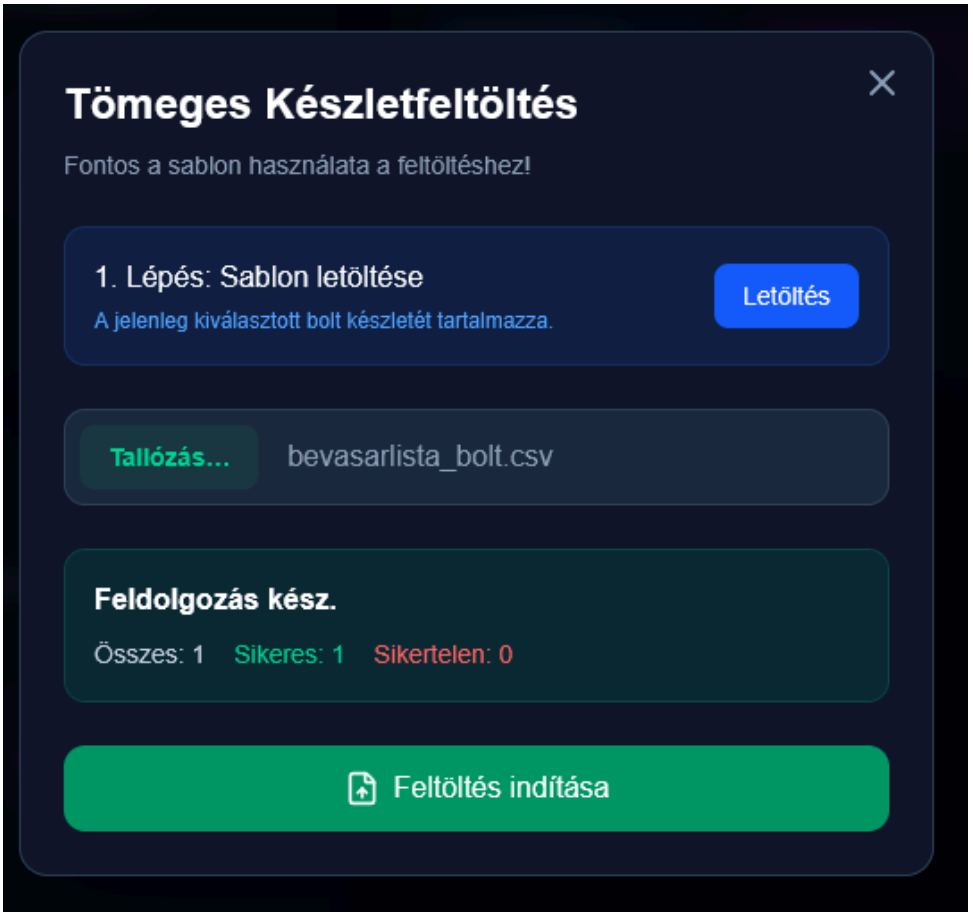

Ha mindez meg van, akkor a zöld feltöltés gombra kattintva megjelenik egy felugró menü, ami elkéri a .csv kiterjesztésű fájlt (a fájl neve nem fontos, csak a kiterjesztése):



Adjuk meg a megváltoztatott .csv fájlunkat, majd nyomjunk rá a ‘Feltöltés indítása’ gombra:



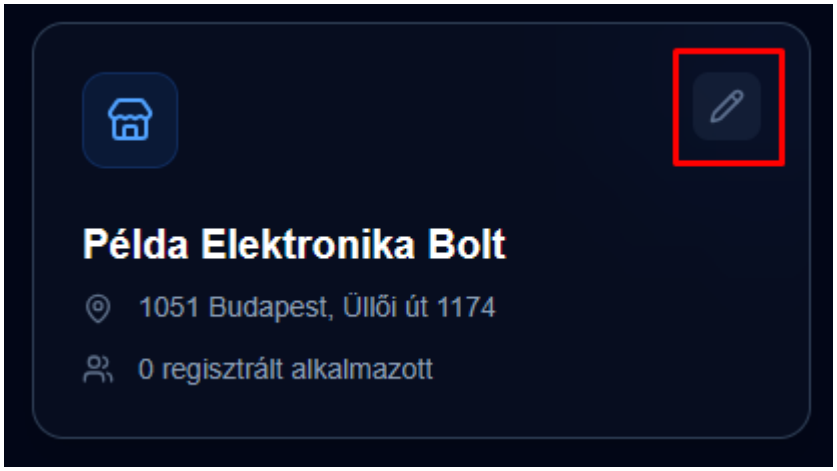

Egy kisebb pontban megmutatja, hogy hány sort sikerült megváltoztatni, hány sort nem:



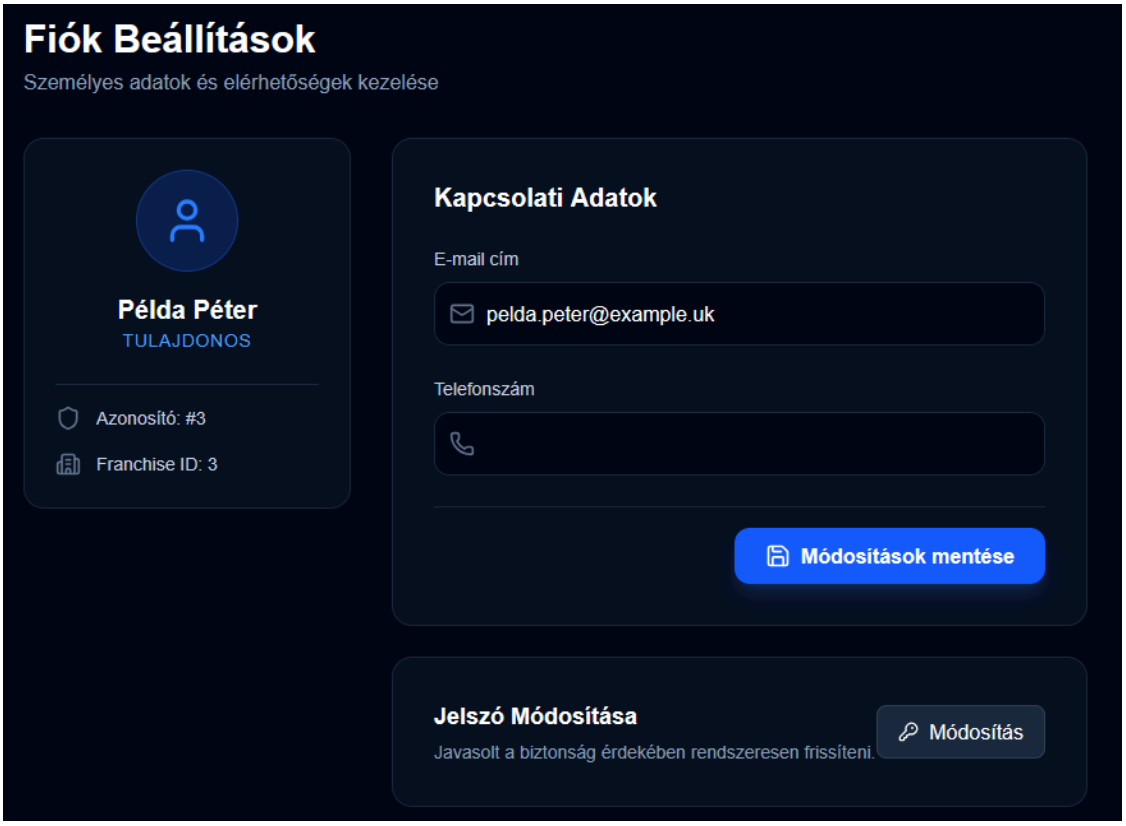
Ha újra megnézzük a Fanta Narancs 1L üdítőitalnak a Raktáron rejlő számát, akkor láthatjuk, hogy a megfelelő 30 darabot sikeresen hozzáadta:

A **‘Eladásoknál’** nagy különlegességek nincsenek, csak az, hogy itt is lehet törölni a piros kuka ikonra rányomva. Itt viszont nem lehet módosítani adatot.

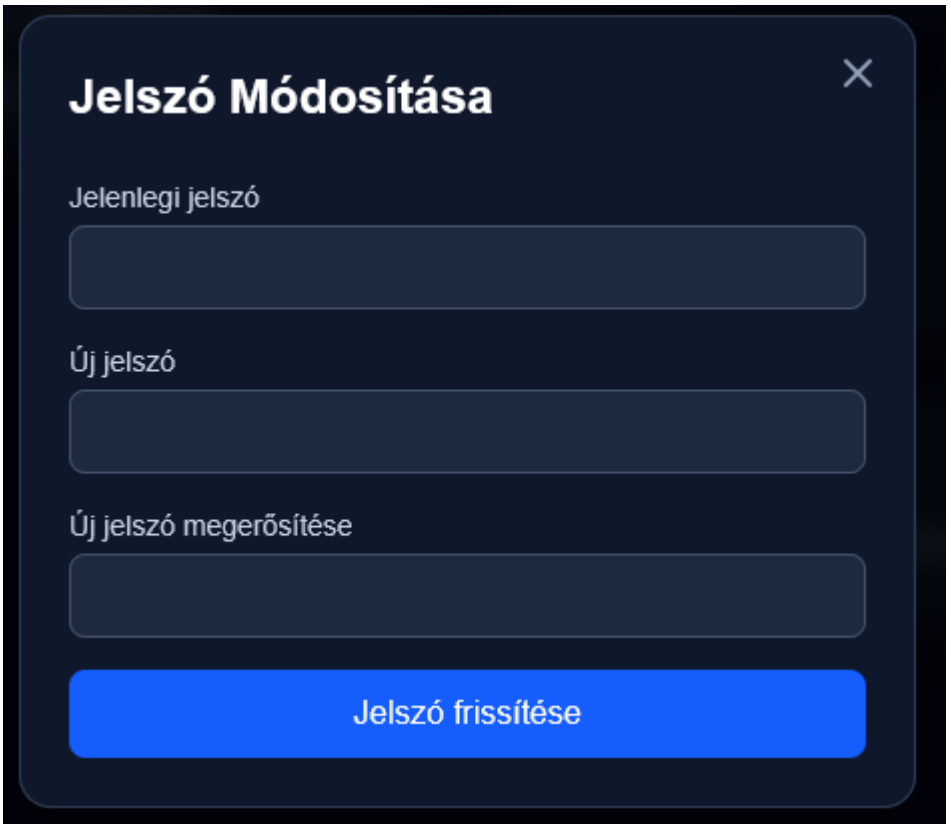
A **‘Boltok’** oldal kizárólag az üzletlánc tulajdonos jogkörrel rendelkező fiókoknál fog megjelenni, és itt csak annyi, hogy akkor jelenik meg a módosítás gomb, amikor ráhúzzuk az egeret.




A **‘Fiók’** az egyetlen oldal, ami minden jogkörnél megjelenik. Itt található a fiókkal kapcsolatos fontos információk változtatása, mint például az email cím, telefonszám és jelszó módosítás lehetősége.



A jelszó módosításhoz felugró menü így néz ki:

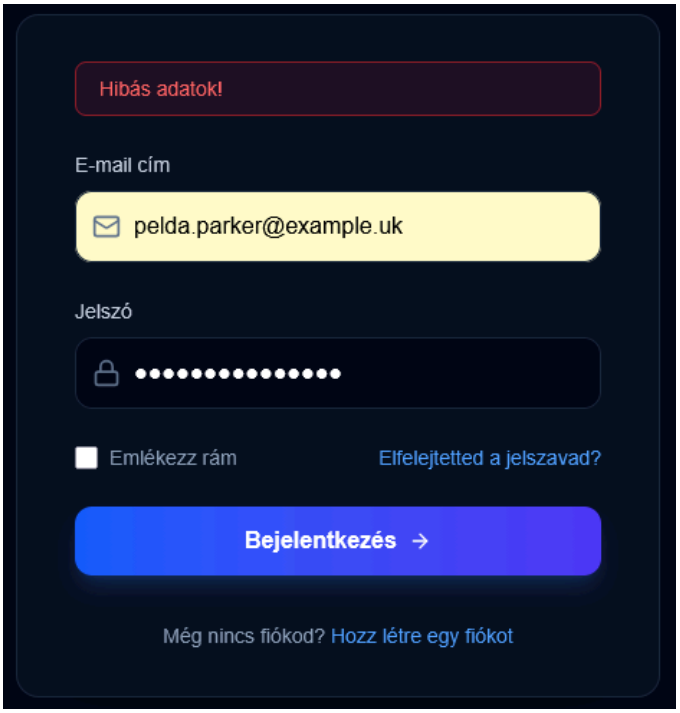



Ezeket a műveleteket akárhányszor meg lehet ismételni.

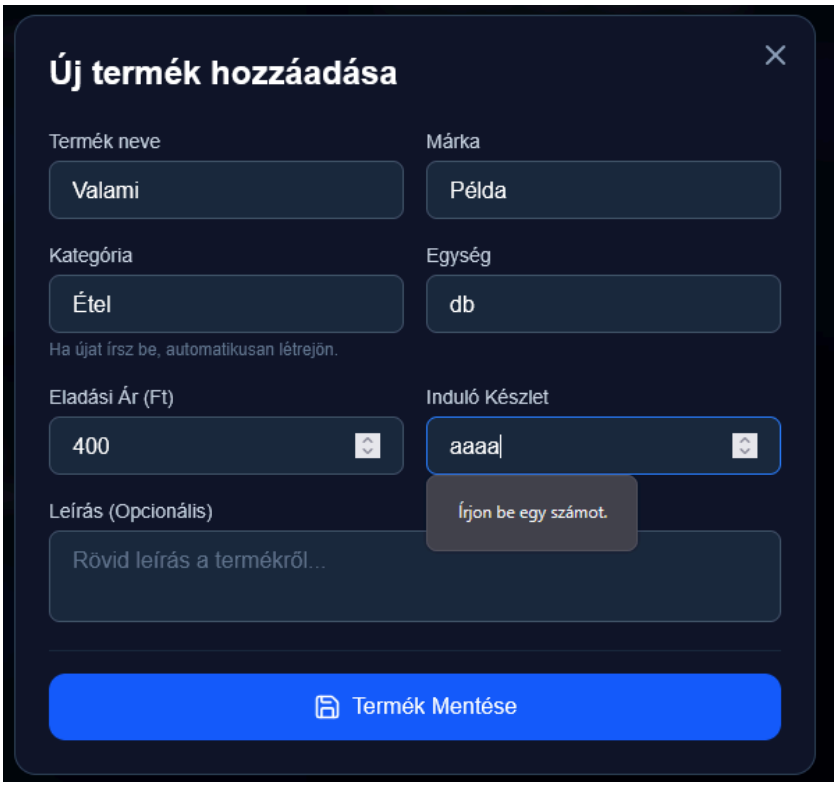
## 3.5. Hibajelzések

A hibajelzések oldaltól függően változhat kinézetével és üzenetével. Van, ahol egy lekerekített téglalap alakú vörösen megjelenő mező lesz, máshol pedig egy egyszerű szürke hátterű megjelenő mező fog megjelenni, amiben egy rövid mondatba sorolja a hibát egy generikus módszerrel. Egy-két példa a hibaüzenetkre és a kinézetre:

**Autentikációs hiba** (elírt jelszó példa)



**Adatbeviteli hiba** (számot vár)




## 3.6. Mit lehet elérni Regisztráció nélkül?

Igazából az összes főprogram részek semelyikét se lehet elérni regisztráció vagy bejelentkezés nélkül, hiszen az egész program arra épül fel, hogy van az illetőnek üzletlánca, vagy legalább egyetlen boltja, amit lehet digitálisan kezelni. A két kritérium hiánya esetén a program fő részeit nem lehet elérni és csak a fő oldalt tudják meglátogatni információért.

## 3.7. Program jogkörök részletesen

A programunk a dokumentáció dátumának írása közben 4 jogkört használ választóvonalakként a 4 különböző munkák között: (Áruházlánc) Tulajdonos, Üzletvezető, Eladó és Készletkezelő.

Mindegyik jobkörnek van egy megadott szerepe a programban, hogy mit lehet elérni és kezelni és mit nem lehet. A jogkörök az AuthLevel táblában találhatóak és két kategóriák van sorolva:

- Admin, vagy teljes jogosultságú jogkörök, akik a hatáskörükben szinte mindent megtudnak változtatni vagy kezelni manuálisan: **(Áruházlánc) Tulajdonos**, **Üzletvezető.**
- Specifikus munkákat ellátó, szűk jogosultságokkal rendelkező jogkörök: **Eladó**, **Készletkezelő**.

A 4 jogkörnek a megadott jogkörrel elvégezhető műveletek konkrét listája:

- **(Áruházlánc) Tulajdonos (ID: 1)**: Minden oldalhoz (Alkalmazottak, Termékek és Eladások) teljes joga van lekérni, hozzáadni, változtatni vagy törölni adatokat. Továbbá, hozzáférést kap egy ötödik oldalhoz, a 'Boltok' oldalhoz, ahol új boltokat tud felvenni, vagy módosítani, vagy törölni az áruházláncához és aközött tud váltogatni, hogy melyik boltot szeretné kezelni abban az időben.
- **Üzletvezető (ID: 2)**: Minden oldalhoz (Alkalmazottak, Termékek és Eladások) teljes joga van lekérni, hozzáadni, változtatni vagy törölni adatokat. Az áruházlánc tulajdonossal ellentétben ez a jogkör csak a megszabott boltot tudja kezelni, a 'Boltok' oldalhoz nincsen hozzáférése.
- **Eladó (ID: 3)**: Csakis specifikusan az 'Eladások' oldalhoz van hozzáférése, ahol csak hozzáadni tud a gyűjteményhez. Habár törölni tud, csak az saját maga által írt eladás adatokat tudja.
- **Készletkezelő (ID: 4)**: Csakis specifikusan a 'Termékek' oldalhoz van hozzáférése, ahol csak hozzáadni tud a gyűjteményhez és módosíthatja csakis csak a raktáron lévő készlet mennyiségét.

**Fontos tudnivaló:** Ez a lista változhat a jövőben.

## 3.8. Elérhetőségek információ kérés esetén

A következő Email címek valamelyikére küldve meg tud minket találni információ szerzési okok iránt:

- **Princz Dániel Olivér** - princz.daniel.oliver@ganziskola.hu
- **Baranyi-Bujdosó Gergő** - baranyi-.bujdoso.gergo@ganziskola.hu

# 4. Összefoglalás

## 4.1. Munkánk értékelése

Őszintén vélve büszkék vagyunk magukra, hiszen Junior programozóként egy ilyen komplett és elég komplex programot életre bírni, főleg egy 2 emberes csapatnál hatalmas munka, de mégis sikerült egy épkézláb programot megírni. De a tényeket nem fogom semmibe venni, hogy nagy fába vágtuk a fejszénket és lehetett volna több is, ha jobban osztjuk be az időnket és nem lettünk volna egy kicsit az u.n. túlterjeszkedés áldozataivá válni, mert többször is át kellett írni bizonyos részeket, kevés tudással (a keretrendszerek szüksége miatt) kezdtük el és többször voltak vitáink a program dizájnja és felépítése kapcsán.

Szerintem ez a hatalmas lecke volt számunkra, hogy hogyan kellene beosztani az időnket megfelelően és hogy jobban kéne felbecsülni a munka terhét egy ilyen mértékű programnál.

## 4.2. Hogyan hasznosulhat a továbbiakban?

Mivel egy célra lett kitalálva ez a program, hogy üzletlánc és bolt tulajdonosoknak adjunk egy közös adminisztrációs rendszert, így a továbbiakban csak így látjuk hasznosulását. Nehéz igazán megmondani, hogy hogyan lehetne még ezt a programot bővíteni új célközönségekhez, de a fejlesztési lehetőségek (2.4.) résznél az integrációs fejlesztések lehetnek azok a részek, amikkel még több embernek tud a hasznára lenni.

Jelenlegi állapotát nézve, csak üzletlánc és bolt tulajdonosoknak, esetleg még az alkalmazottjaiknak lehet még valami haszna a programnak.

# 5. Irodalmi Jegyzetek

Itt található a megfelelő linkek a dokumentumban, amiket a programhoz használtunk mint információforrás, vagy írott segítség:


- https://nodejs.org/en/download (Node.Js letöltő link)
- https://www.npmjs.com/package/validator (Validátor csomag)
- https://www.npmjs.com/package/jwt-decode (JSON WebToken dekódoló csomag)
- https://www.npmjs.com/package/jsonwebtoken (JSON WebToken csomag)
- https://www.npmjs.com/package/express (Express csomag)
- https://www.npmjs.com/package/mssql (Microsoft SQL értelmező csomag)
- https://www.npmjs.com/package/dotenv (.Environment titkosító csomag)
- https://www.npmjs.com/package/bcrypt (Jelszó hashelő csomag)
- https://www.npmjs.com/package/multer (Fájl kezelő és beolvasó csomag)
- https://www.npmjs.com/package/iconv-lite (Fájl konvertáló csomag)
- https://www.npmjs.com/package/lucide (Lucide alap ikonokhoz csomag)
- https://github.com/DaniDay22/AssetHub (A Program GitHub repója)
- infojegyzet.hu/webszerkesztes/dokumentacio/ (Útmutatás a megfelelő Dokumentáció elkészítéséhez)
- https://code.visualstudio.com/download (Visual Studio Code letöltő link)
- https://learn.microsoft.com/en-us/ssms/release-history (SQL Server Management Studio 2021 letöltő link)